MCMC Part 1

Julia Wrobel

Overview

Today, we cover:

- Markov chains
- MCMC
- Metropolis-Hastings

Announcements:

- Next week spring break!
- No class Wednesday, 3/18 (ENAR)
- Resume Bayesian Computing on 3/19
- Look out for email from Keven Haynes about the cluster

Resources:

- Peng Advanced Stat Computing Chapter 7

Joint posterior distribution

Our goal is to learn about the joint posterior distribution

$$p(\beta, \sigma_y^2 | y, X, hyperparameters)$$

This can be hard!

- Often the joint posterior is analytically intractable
- For the conjugate priors we use, though, the full conditionals were "easy"
- If we can't write down the joint posterior, maybe we can sample from it

MCMC

Basic idea: How do I get a string of values from the posterior when I cannot write the posterior down?

- Markov chain Monte Carlo (MCMC) methods let you sample from complicated [posterior] distributions
  - a collection of tools used to sample from a target distribution
- Once you have a sample from the posterior distribution you can use summary statistics (mean, variance, quantiles) to do posterior inference

First we will review Markov chains

Markov chains

A Markov chain is a stochastic process that evolves over time by transitioning into different states.

- The sequence of states is denoted by X_i
- Transition between states is random and follows the *Markov property*:

$$P(X_t|X_{t-1}, X_{t-2}, \dots, X_0) = P(X_t|X_{t-1})$$

FYI: this notation is for discrete distributions

Markov chains

- **Core concept**: The probability of transitioning to a state at time t depends only on the most recent previous state
- *State space*: collection of states that a Markov chain can visit
- *Transition matrix*: collection of probabilities that we move from one state to another

Markov chain example

Markov chain properties

$$P = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{bmatrix}$$

Each entry of P is the probability of going from state i to state j, i.e.

$$P(X_{n+1} = j | X_n = i) = P_{ij}$$

- Sum of weights from outgoing arrows $= 1$
- Starting with one state and sampling a string of subsequent states is called a **random walk**
- Goal is to find probabilities of being in each state

Markov chain properties

Suppose we start the chain in state 3, with probability 1, so the initial probability distribution over the three states is $\pi_0 = (0, 0, 1)$.

- What is the probability distribution over the states after one iteration?

- What is the probability distribution over the states after n iterations?

π_* is called the stationary distribution. This is the vector of probabilities of being in each state that the chain converges to

Markov chain limit theorem

Basic limit theorem for Markov chains says

$$||\pi_* - \pi_n|| \to 0 \text{ as } n \to \infty$$

where $|| \cdot ||$ is the total variation distance between two densities.

- another way of saying this is $\lim_{n\to\infty} \pi^n = \pi_*$ regardless of the starting state

In the long run, the chain "forgets" where it started and settles into a stable distribution over the states. That stable/stationary distribution is π_*, and the process spends a proportion π_j of time in state j.

Markov chain assumptions

(1) The stationary distribution π_* exists.

(2) The chain is **irreducible**: it is possible to reach any state from any other state (eventually)

(3) The chain is **aperiodic**: the chain does not cycle in a fixed period. A chain is aperiodic if it does not make deterministic visits to a subset of the state space.

Time-reversible Markov chains

In a reversible Markov chain, the process looks the same if you run it forward or backward in time.

- At equilibrium (the chain's long-run steady state), the traffic between any two states is perfectly balanced. For any two states A and B, the probability of transitioning from A to B times the long-run probability of being in A equals the probability of going from B to A times the long-run probability of being in B

Why Reversibility Matters for MCMC

You want samples from $p(|y) = $ complicated posterior distribution. Sampling directly is hard. Instead:

1. Build a Markov chain
2. Ensure stationary distribution = target distribution
3. Run the chain for a while.
4. Visited states become samples from the target.

MCMC methods use Markov chains to sample from a target probability distribution. The goal is to construct a chain whose stationary distribution is the target distribution we want to sample. Reversibility ensures we have the correct stationary distribution.

Why Reversibility Matters for MCMC

- Ensures correctness: target distribution is the stationary distribution
- Prevents systematic bias: no net directional drift in sampling
- Simplifies algorithm design and analysis (e.g., Metropolis-Hastings)
- Makes it easier to combine steps while preserving invariance
- Widely used in MCMC due to theoretical guarantees and reliability

Review: rejection sampling

MCMC intuition

- Next sample depends on the last sample
- Samples are no longer independent
- Eventually Markov chain will simulate draws from target distribution
- If a Markov chain reaches a stationary distribution, it will stay there
  - Goal is to design a Markov chain whose stationary distribution is the target distribution

MCMC

Term MCMC encompasses broad array of techniques that have in common a few key ideas:

1. We want to sample form a complicated density or probability mass function π, often a posterior distribution. Assumption is we can evaluate π but not *directly* sample from it
2. We know that certain stochastic processes called *Markov chains* will converge to a stationary distribution.

- Simulating from a Markov chain for long enough will eventually gives us a sample from the chain's stationary distribution

3. Given the functional form of the density π, we want to construct a Markov chain that has π as its stationary distribution
4. We want to sample values from the MC such that the sequence of values $\{x_n\}$ generated by the chain converges in distribution to π.

Metropolis-Hastings algorithm

Suppose we want to take a sample $x \sim \pi(x)$ using MCMC.

- Metropolis-Hastings is a popular algorithm (circa 1970) for doing this.

Some terminology:

- **target distribution** is $\pi(x)$, the distribution I want to sample from
- **proposal distribution** is $q(y|x)$, some probability distribution that is easy to sample from that has the same support as π
    - You can equivalently think of this as a transition probability to get from x to the next state y
- **acceptance probability** aka **acceptance ratio**, denoted $\alpha(y|x)$, is the probability of transitioning to a new stage in the Markov chain

Steps of Metropolis-Hastings

The Metropolis-Hastings procedure is an iterative algorithm where at each iteration, there are three steps. Suppose we are currently in state x and want to know how to move to the next state in the state space (which we will call y).

(1) Simulate a *candidate* value $y \sim q(y|x)$ from the proposal distribution, conditional on current state x.

(2) Calculate the acceptance ratio,

$$\alpha(y|x) = \min\left\{\frac{\pi(y)q(x|y)}{\pi(x)q(y|x)}, 1\right\}$$

(3) Simulate $u \sim Unif(0, 1)$. If $u \leq \alpha(y|x)$, then the next state is equal to y. Otherwise, the next state is still x.

Simple example

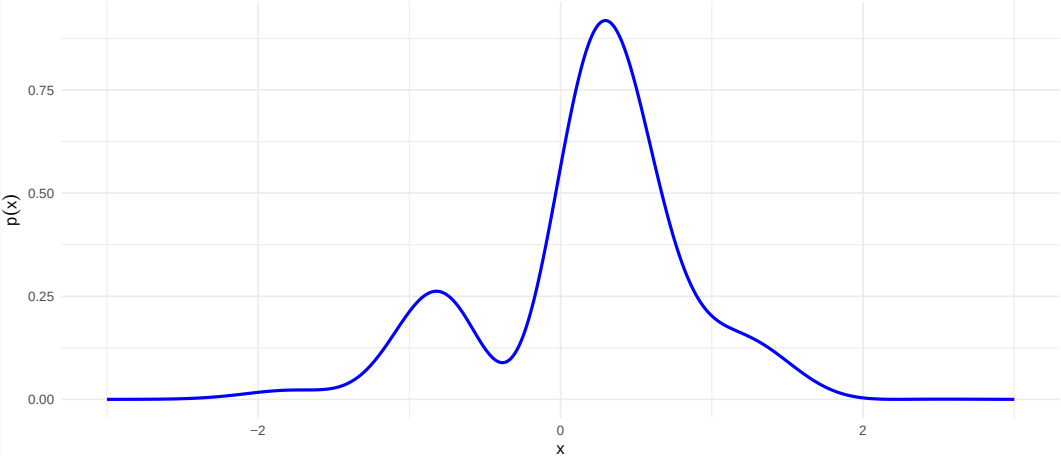
Say we want to sample from the probability distribution $p(x)$ given below:

$$p(x) = \frac{\exp(-x^2)(2 + \sin(5x) + \sin(2x))}{\int_{-\infty}^{\infty} \exp(-x^2)(2 + \sin(5x) + \sin(2x))dx} \tag{1}$$

$$\propto \exp(-x^2)(2 + \sin(5x) + \sin(2x)) \tag{2}$$

Simple example

$$p(x) = \frac{e^{-x^2}(2 + \sin(5x) + \sin(2x))}{Z}$$



Simple example

Domain of x is unrestricted. Let x be our current value and y the next value in the chain.

- To generate y we will add a $N(0, 1)$ random variable to our current value of x
  - This implies the proposal distribution $q(y|x)$ follows $y \sim N(x, 1)$

The more general idea here is to let $y = x + \epsilon$, where $\epsilon \sim g$ and g is a probability density symmetric about 0.

- Sometimes called a "random walk" Metropolis-Hastings
- Using a symmetric proposal distribution is also called the Metropolis algorithm

Acceptance ratio step

$$\alpha(y|x) = \min\left\{\frac{\pi(y)q(x|y)}{\pi(x)q(y|x)}, 1\right\}$$

Acceptance ratio step

$$\alpha(y|x) = \min\left\{\frac{\pi(y)}{\pi(x)}, 1\right\}$$

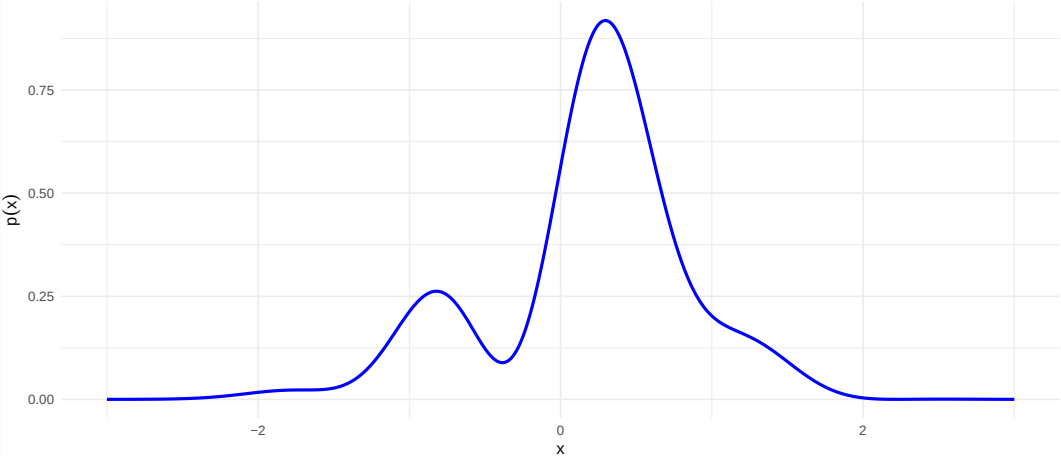
First, we calculate this ratio.

Then simulate $u \sim Unif(0, 1)$.

- If $u \leq \alpha(y|x)$, then the next state is equal to y. Otherwise, the next state is still x.

Simple example

$$p(x) = \frac{e^{-x^2}(2 + \sin(5x) + \sin(2x))}{Z}$$



Simple example implementation

```r
target = function(x){
  exp(-x^2) * (2 + sin(5 * x) + sin(2 * x))
}

metropolis_step = function(x, sigma) {
  y = rnorm(1, mean = x, sd = sigma)
  accept_prob <- min(1, target(y) / target(x))
  u = runif(1)
  if(u <= accept_prob) {
    value = y
    accepted = TRUE
  } else {
    value = x
    accepted = FALSE
  }
```

Simple example implementation

```
metropolis_sampler = function(initial_value,
                              n = 1000,
                              sigma = 1) {

  results = as.list(rep(NA, n))
  current_state = initial_value
  for(i in 1:n) {
    out = metropolis_step(current_state, sigma)
    current_state = out$value
    results[[i]] = out
  }
  results = do.call(rbind, results)
  results
}
```

Simple example implementation

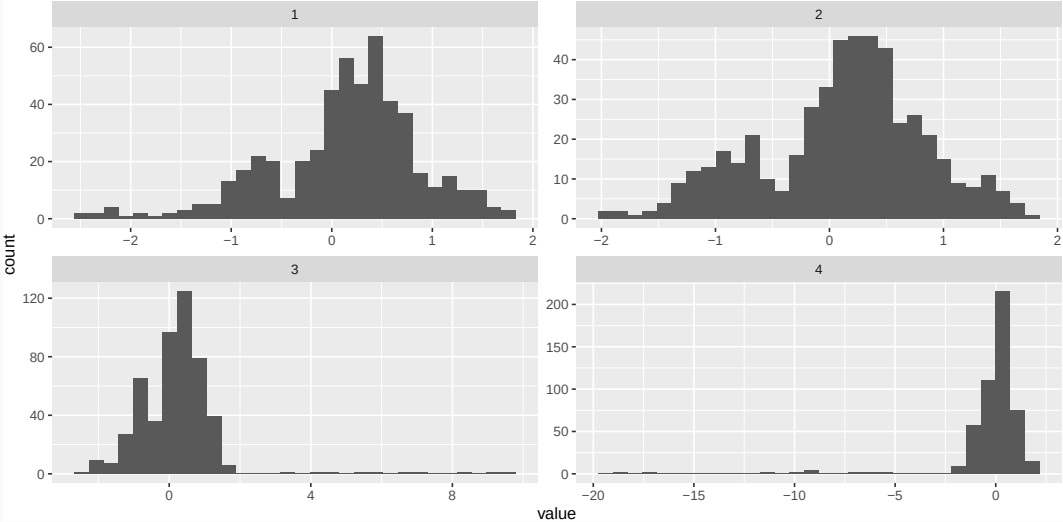
```r
set.seed(34634634)
chain1 = metropolis_sampler(initial_value = 0, n = 1000)

head(chain1)
```

```
# A tibble: 6 x 2
    value accepted
    <dbl> <lgl>
1 -0.0264 TRUE
2 -0.0791 TRUE
3 -0.0791 FALSE
4 -0.0791 FALSE
5  1.46   TRUE
6  1.46   FALSE
```

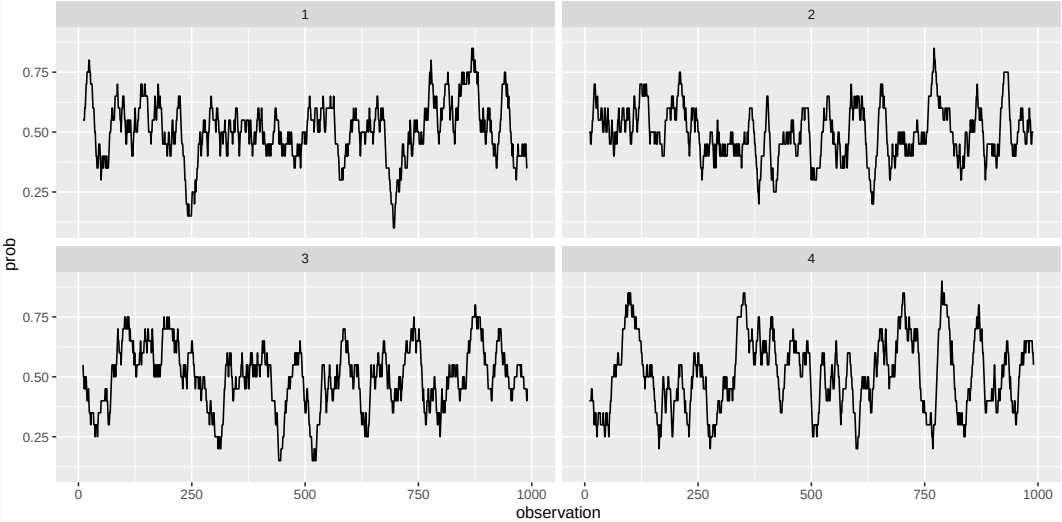
Visualizing results

```r
set.seed(3463)
chain2 = metropolis_sampler(initial_value = 0, n = 1000)
chain3 = metropolis_sampler(initial_value = 10, n = 1000)
chain4 = metropolis_sampler(initial_value = -20, n = 1000)
```

Visualizing results



Acceptance probabilities



Some terminology

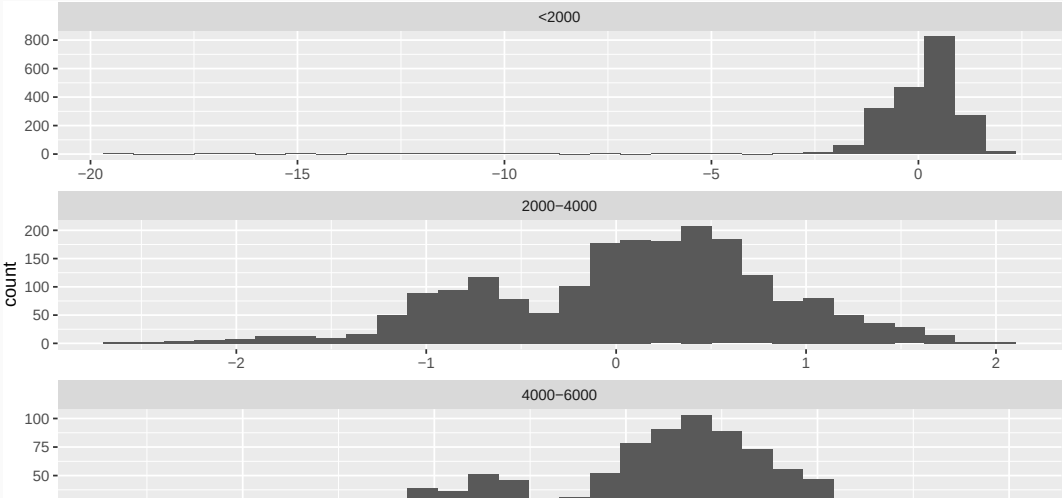
- **Burn-in**: the initial period of an MCMC simulation discarded to allow the chain to reach its stationary distribution.
- **Lag**: refers to the number of steps between samples used to reduce autocorrelation and improve the independence of the retained draws.
- **Mixing rate**: refers to how quickly an MCMC chain explores the target distribution, with better mixing indicating faster decorrelation between samples.
- **Proposal width** is a parameter of the proposal you can vary to tune this

Burn-in

Convergence occurs regardless of our starting point (in theory), so we can usually pick any reasonable values in the parameter space for initial value(s).

- The farther away you are from high density regions the longer it will take to reach the stationary distribution (i.e. target distribution $\pi(x)$)
- Samples you generate early on will probably not follow $\pi(x)$
- Generally we throw out a certain number of the first draws, known as **burn-in** to make sure our sample is closer to the stationary distribution and less dependent on starting values

Burn-in: $x_0 = -20$

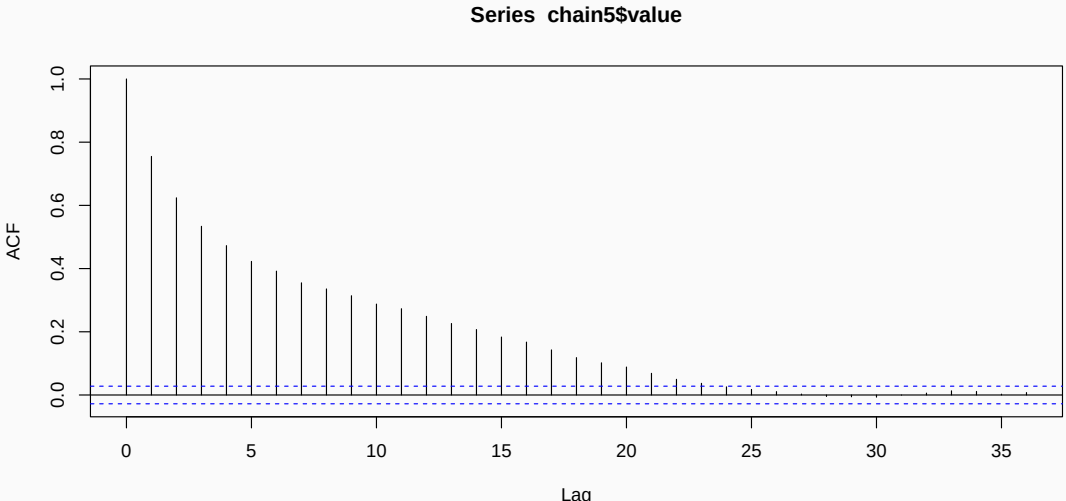

<2000
800
600
400
200
0
−20
−15
−10
−5
0
2000−4000
200
150
100
50
0
−2
−1
0
1
2
4000−6000
100
75
50
count

Lag

This is essentially the number of iterations we run the sampler for between successive samples.

- MCMC samples are not independent! By definition, each sample depends on the previous sample
    - This means there is a time-dependent autocorrelation
- One way to reduce correlation in your samples and have draws that are more independent is to allow several iterations of the sampler to elapse in between successive samples.
- This is the lag
- More of a problem when the MCMC cannot make big jumps within the parameter space

Lag: autocorrelation plots



Mixing rate

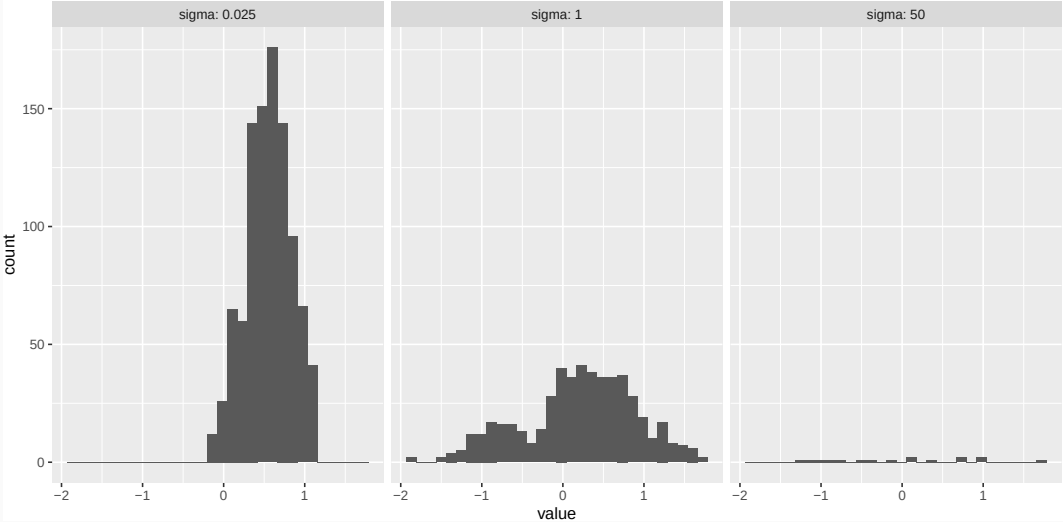
In practice the choice of proposal distribution matters a lot.

- If it's too narrow, your sampler will have an extremely high acceptance rate on average, but it will move around extremely slowly, that is,
  - i.e. it has a low *mixing rate*.
- If it's too wide, the acceptance rate becomes too low and the chain gets stuck on specific values for long periods of time.
- Varying the proposal width can affect this

Effect of proposal width

```r
set.seed(34634)
chain_1 = metropolis_sampler(initial_value = 0, n = 1000, sigma = 1)
chain_.025 = metropolis_sampler(initial_value = 0, n = 1000, sigma = .025)
chain_50 = metropolis_sampler(initial_value = 0, n = 1000, sigma = 50)
```

Effect of proposal width



Effect of proposal width

Convergence

We will not cover the convergence theory behind Metropolis chains in detail, but ...

- The Markov process generated under this procedure is ergodic (irreducible and aperiodic) and has a unique limiting distribution (stationary distribution).
- E rgodicity means that the chain can move anywhere at each step, which is ensured if $q(y|x) > 0$ everywhere
- By construction, Metropolis chains are reversible, so that $\pi(y|x)$ is the stationary distribution

Convergence diagnostics

Convergence diagnostics are used to assess if your chain has converged.

- Cannot guarantee if a chain has converged, but they can indicate that it has not converged
- Not specific to Metropolis-Hastings, apply to MCMC more generally

- Trace plot
- Autocorrelation plot
- Density plots for multiple chains
- Effective sample size

Trace plot

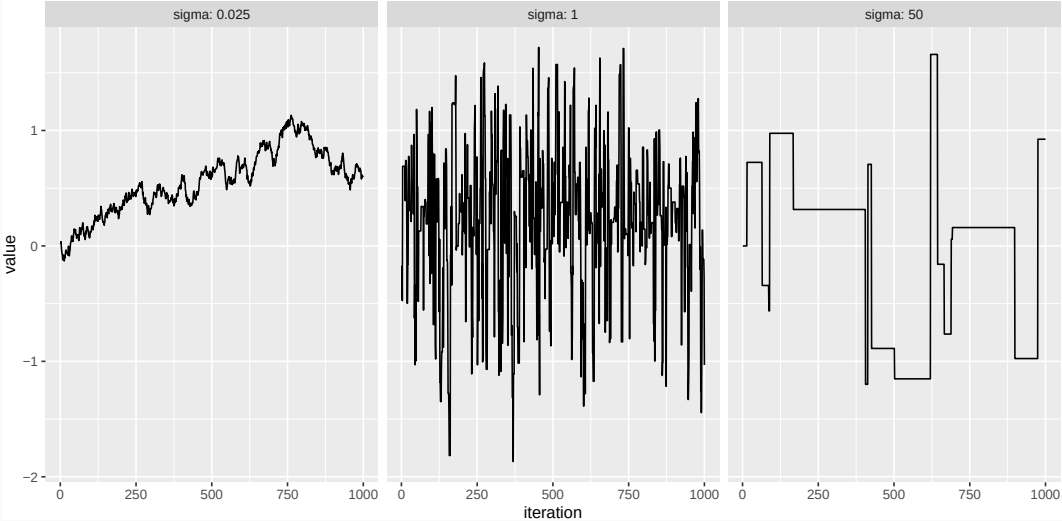
Shows sample values of an MCMC parameter over iterations

- Plot the whole chain (both accepted and rejected samples)

The chain should:

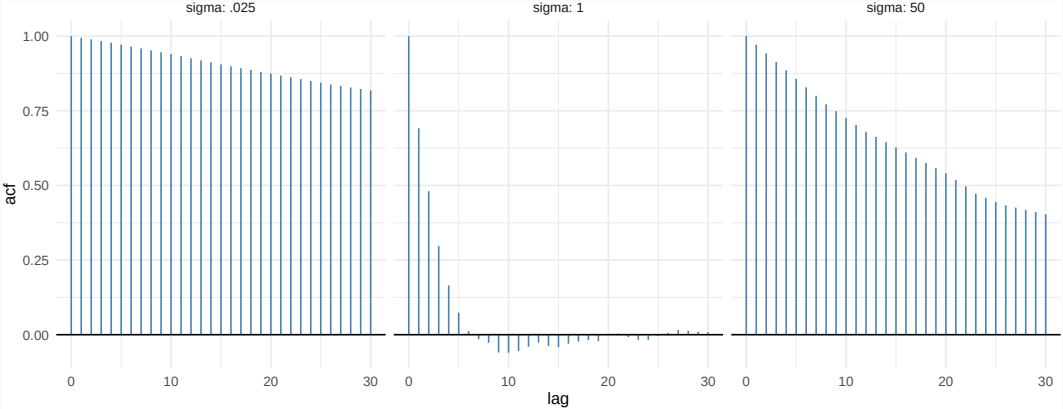
- fluctuate around a stable mean without visible trends or drifts over time
- Avoid long streaks of similar values (these indicate high autocorrelation or poor mixing)
- Explore the full parameter space

Trace plot

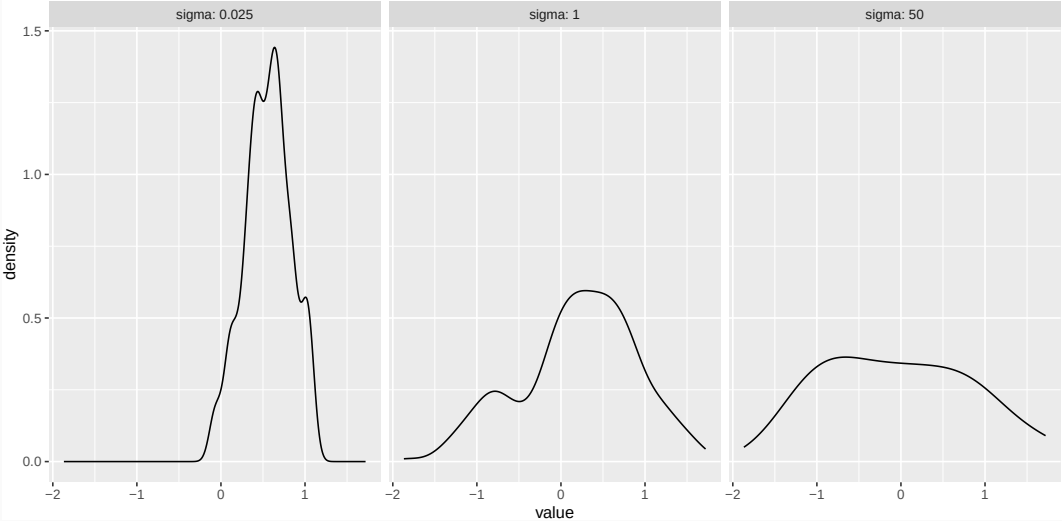

sigma: 0.025
sigma: 1
sigma: 50
value
1
0
−1
−2
0
250
500
750
1000
iteration

Autocorrelation plot

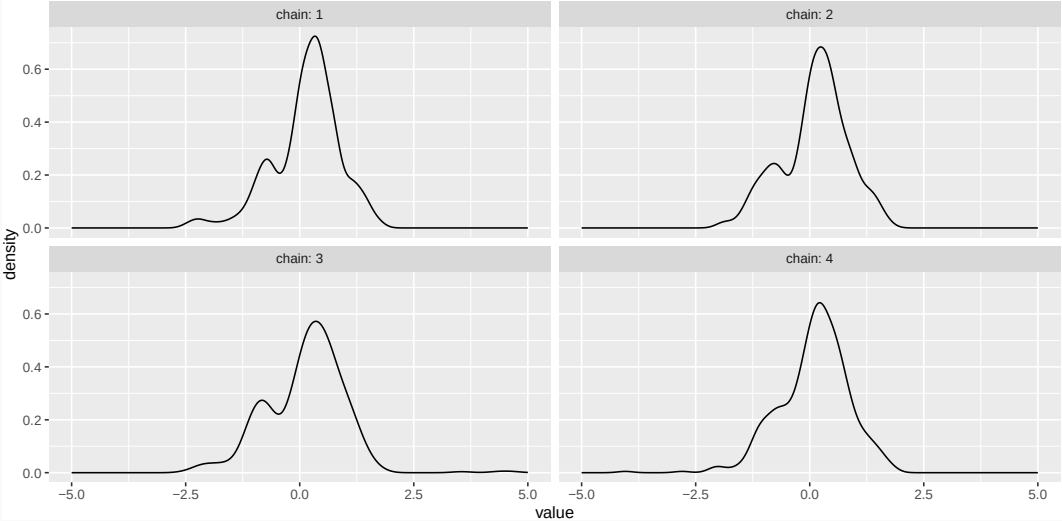
Shows correlation between samples at different lags. Lower autocorrelation means better mixing and more efficient sampling.



Density plots



Density plots



Effective sample size

The effective sample size translates the number of MCMC samples S into an equivalent number of independent samples.

$$ESS = \frac{S}{1 + 2\sum_k \rho_k}$$

where ρ_k is the lag k autocorrelation.

- Can calculate ESS using `coda::effectiveSize`.

Effective sample size

```r
library(coda)

effectiveSize(chain_.025$value) %>% as.numeric()
```

```
[1] 2.742461
```

```r
effectiveSize(chain_1$value) %>% as.numeric()
```

```
[1] 207.9327
```

```r
effectiveSize(chain_50$value) %>% as.numeric()
```

General diagnostic rules

- Run multiple chains with different starting values
- Discard the first 1000 or so samples (depending on the problem)
- Check diagnostic plots!
- There are other diagnostic statistics (Gelman-Rubin, Geweke) that we haven't discussed

Resources

- Cute Markov chain example
- Intro MCMC video